SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING **COURSE CODE: CSA4304**

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 20**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
```

```
  <title>TestPage</title>
</head>
  <body>
  <div>
  <p>HelloWorld
  </div>
 </body>
 </html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I **Mohamed Haroon P (192511313)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	24 / 25	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	24 / 25	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	19.2 / 20	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	19.2 / 20	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	9.6 / 10	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

QUESTION 1 — HTTP Response Analysis

Scenario: A web server returns a response header that looks normal at first glance, but two of its fields are invalid.

Given HTTP Response Header

HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive

Q1 — Identify Any Incorrect or Invalid Fields in the Header

Field	Status	Problem
Content-Length: -150	**INVALID**	Content-Length must be a non-negative integer (digits only) giving the body size in bytes. A negative number is impossible and breaks HTTP message framing (RFC 9110).
Date: Mon, 01 Apr 2026 ...	**INVALID**	The IMF-fixdate format requires the weekday to match the calendar date. 01 April 2026 is a **Wednesday**, not a Monday.
Content-Type: text/html	Valid (incomplete)	Syntactically correct, but best practice is to add charset=UTF-8 so browsers do not have to guess the encoding.
Connection: keep-alive	Valid	Correct. Keeps the TCP connection open for further requests. This makes a correct Content-Length even more important (see Q2).

HTTP/1.1 200 OK	Valid syntax	Status line is well-formed (version, code, reason phrase), but see Q3 on whether it is semantically correct.

Q2 — Analyze the Issue with Content-Length and Explain Its Impact

What Content-Length does: it tells the client exactly how many bytes of body to read. With keepalive, the same TCP connection carries many responses back to back, so Content-Length is the only way the browser knows where one response ends and the next begins.

Why -150 is wrong: a body cannot have a negative number of bytes. The grammar for the header is Content-Length = 1*DIGIT, so the minus sign makes the field syntactically invalid.

Impact	Explanation
Response rejected	Per RFC 9112, an invalid Content-Length makes the message framing unrecoverable. Browsers typically show an error (e.g. *ERR_INVALID_CONTENT_LENGTH*) or discard the response.
Connection must be closed	The client cannot tell where the body ends, so it must close the connection. This defeats the purpose of keep-alive and wastes the TCP handshake.
Hanging / truncated page	A client that guesses may wait forever for bytes that never come, or cut the page short, leaving blank or half-rendered content.
Security risk	Inconsistent length handling between a proxy and a server enables **HTTP response splitting / desynchronisation** attacks, so strict parsers reject such messages deliberately.

Q3 — Does the Status Code Match the Response Correctness?

No. The code 200 OK promises that the request succeeded and a valid representation follows. But the header the server actually produced is malformed, so the response is **not** a correct success response.

- A 200 status only tells the client *what happened*; it cannot repair a broken header. The status and the headers contradict each other.
- If the server genuinely could not build a valid response, the honest status would be **500 Internal Server Error** (server-side bug).

- If the body is fine and only the header generation is buggy, 200 is the correct code *after* the headers are fixed. So,the code is right in intent, but wrong in practice until the errors are corrected.

Q4 — Suggest Corrections for the Identified Errors

Corrected HTTP Response Header

HTTP/1.1 200 OK

Date: Wed, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html; charset=UTF-8

Content-Length: 150

Connection: keep-alive

Error	Fix
Content-Length: -150	Set it to the exact byte count of the body (e.g. 150). If the size is unknown in advance, remove it and use Transfer-Encoding: chunked instead (never send both).
Date weekday mismatch	Generate Date automatically from the system clock in GMT using the server framework, never hard-code it: Wed, 01 Apr 2026 10:00:00 GMT.
Missing charset	Use Content-Type: text/html; charset=UTF-8.
Prevention	Let the web server or framework (Nginx, Apache, Express, Spring) compute headers, and test responses with curl -i or browser DevTools → Network tab.

Key Takeaway: Two fields are invalid: **Content-Length: -150** (must be a non-negative byte count) and the **Date** weekday (01 Apr 2026 is a Wednesday). A malformed header makes a 200 OK untrustworthy, so the fix is a correct byte length, a correct date and a charset.

QUESTION 2 — DOM Structure Analysis

Scenario: A small page is written with an unclosed paragraph and no document type declaration.

Given Code

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

Q1 — Identify Missing or Improperly Closed Elements

Issue	Detail
never closed	The HelloWorld has no . It is the only improperly closed element.
Missing <!DOCTYPE html>	No doctype, so browsers enter **Quirks Mode**.
Missing lang on <html>	Needed by screen readers and translation tools.
Missing <meta charset>	No character encoding declared.
Missing viewport meta	Mobile browsers will render at desktop width.
Text "HelloWorld"	No space between the words; the content is almost certainly meant to be *Hello World*.

Note: unlike XHTML, HTML5 allows the end tag of to be omitted in some positions, so the file is not a fatal error, but it is still bad practice and not well-formed XML/XHTML.

Q2 — Analyze How the Browser Will Interpret This Structure

The browser's HTML parser follows the WHATWG error-recovery algorithm. When it meets

 while a <p> is still open, it **implicitly closes the paragraph first**, then closes the div. The resulting DOM tree is:

DOM Tree Built by the Browser

```
html
 +-- head
 |   +-- title  -> "TestPage"
 +-- body
     +-- div
         +-- p  -> "HelloWorld"      (auto-closed by the parser)
```

- The **DOM is correct in modern browsers**, because the parser silently repairs the error. The page still displays "HelloWorld".
- Because there is no DOCTYPE, the browser selects **Quirks Mode**, so the repaired DOM is styled with legacy rules.
- The page is still "invalid" by the W3C validator; browsers tolerate it, validators do not. **Q3 — Determine the Impact on Rendering and Layout**

Area	Impact
Box model	In Quirks Mode some engines apply legacy width/height/padding calculations, so element sizes differ from Standards Mode.
Default font and table sizing	Font inheritance in tables and body text behaves like 1990s browsers, giving inconsistent text size across browsers.
Paragraph spacing	The <p> gets its default 1em top and bottom margins; this renders fine here because the parser closed it in the right place.
Larger pages (risk)	If more content followed the unclosed <p>, the auto-close position could change and shift layout, which is why explicit closing tags matter.
Accessibility and SEO	No lang attribute and no landmark elements (main, header) reduce screen-reader and search-engine understanding.

Q4 — Suggest Corrections to Make the DOM Valid

Corrected, W3C-Valid HTML5

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TestPage</title>
</head>
<body>
  <div>
    <p>Hello World</p>
  </div>
</body>
</html>
```

- Added **<!DOCTYPE html>** → Standards Mode.
- Closed the paragraph with ** ** → well-formed nesting: html > body > div > p.
- Added **lang**, **charset** and **viewport** → accessibility, correct encoding, mobile support.
- Fixed the text to **Hello World**. Validate the result at validator.w3.org.

Key Takeaway: The only unclosed element is ** **; the parser auto-closes it so the DOM is still built correctly, but the missing **DOCTYPE** triggers Quirks Mode. Closing every tag and adding the doctype makes the DOM valid and the rendering predictable.

QUESTION 3 — GET vs POST in Login System

Scenario: A login system is compared using two HTTP methods.

Method	URL Example	Data Visibility	Security Level

GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Q1 — Interpret the Differences in Data Transmission Between GET and POST

GET request: data in the URL (query string)

```
GET /login?user=admin&pass=1234 HTTP/1.1
Host: app.example.com
(no body)
```

POST request: data in the request body

```
POST /login HTTP/1.1
Host: app.example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 24

user=admin&pass=1234
```

Feature	GET	POST
Where data travels	In the URL query string	In the request body
Purpose (per HTTP spec)	Retrieve a resource (safe, idempotent)	Send data to create / process something (not idempotent)
Visibility	Shown in address bar, history, bookmarks	Not shown in the URL

Logged in server / proxy logs	Full URL with data is logged	Only URL logged, body usually is not
Size limit	Limited by URL length (about 2,000 chars in practice)	No practical limit
Bookmarkable / cacheable	Yes	No (by default)
Browser Back / Refresh	Safe to repeat	Browser warns about resubmitting
Encryption	None by itself	None by itself (HTTPS needed for both)

Q2 — Analyze Which Method Is More Suitable for Login and Why

POST is the correct choice for login.

- A login *submits credentials* to start a session, which changes server state. It is not a plain data lookup, so it matches the intent of POST, not GET.
- POST keeps the password out of the URL, so it does not leak into browser history, bookmarks, the Referer header or server access logs.
- POST responses are not cached or bookmarked by default, so credentials are not stored by intermediaries.
- **Important caveat:** POST is only "Moderate" secure (as the table says). The body is still plaintext on the wire, so **HTTPS is mandatory** to actually protect it.

Q3 — Identify Security Risks in Using GET for Authentication

Risk	Explanation
Credentials in the URL	?user=admin&pass=1234 exposes the password to anyone who sees the screen or address bar (shoulder surfing).
Browser history and bookmarks	The full URL, with password, is stored on the device and may sync across devices.

Server and proxy logs	Web servers, load balancers and proxies log full URLs in plaintext, so every admin with log access sees passwords.
Referer header leakage	When the user clicks an external link, the previous URL (with the credentials) may be sent to the third-party site.
Easy to share by accident	Copy-pasting or screenshotting the URL shares the credentials. Search engines and caches may also index it.
CSRF and replay	GET login links can be triggered by an <img> tag or a link on another site, and replayed from history.

Q4 — Suggest Best Practices for Secure Login Implementation

Secure login form (client side)

```
<form action="/login" method="POST">
  <label for="user">Username:</label>
  <input type="text" id="user" name="user" required autocomplete="username">

  <label for="pass">Password:</label>
  <input type="password" id="pass" name="pass" required autocomplete="current-password">

  <input type="hidden" name="csrf_token" value="{{ server_generated_token }}">
  <button type="submit">Login</button>
</form>
```

- **Use POST** for all credentials, never GET.
- **Serve everything over HTTPS** (TLS) and add the Strict-Transport-Security header.
- **Hash passwords** on the server with a salted, slow algorithm (bcrypt, Argon2); never store plaintext.

- **CSRF token** in the form, validated server-side.
- **Rate-limit and lock out** after repeated failed attempts to stop brute force; consider multifactor authentication.
- **Secure session cookie:** set Secure; HttpOnly; SameSite=Strict after login.
- **Generic error messages** ("Invalid username or password") so attackers cannot discover valid usernames.
- **Validate and sanitise on the server**, since client-side checks alone can be bypassed.

Key Takeaway: GET puts the password in the URL (history, logs, Referer), which makes it unsuitable for authentication. **POST over HTTPS**, with hashed passwords, CSRF protection, rate limiting and secure cookies, is the right login design.

QUESTION 4 — HTML Code Inspection

Given Code

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Q1 — Identify Missing Closing Tags

Element	Problem	Needed
<h1>Welcome	Heading never closed	</h1>

This is a paragraph	Paragraph never closed	
<img>	**No closing tag needed.** It is a void element	Nothing (but needs alt, see Q2)

Other omissions: no lang attribute on <html>, no <meta charset>, no viewport meta, and no <main> landmark.

Q2 — Analyze Issues with the <img> Tag

Issue	Why It Matters
Missing alt attribute	Required by the HTML spec and the W3C validator. Screen readers cannot describe the image, and if the file fails to load nothing meaningful is shown. SEO is also hurt.
Missing width and height	The browser cannot reserve space before the image loads, causing **layout shift (CLS)** when it appears.
It is a void element	<img> has no content and no end tag. Writing </img> is invalid. The self-closing slash <img /> is optional in HTML5 (required only in XHTML).
Generic file name	image.jpg is not descriptive; use meaningful names, and add loading="lazy" for below-the-fold images.
Nested inside the heading (see Q3)	Because <h1> is never closed, the image ends up inside the heading in the DOM.

Q3 — Determine How Browsers Will Handle These Errors

Browsers never refuse to render HTML; the WHATWG parsing algorithm repairs the mistakes. An important detail: a start tag does **not** close an open <h1>, so the paragraph is placed *inside* the heading. The unclosed elements are closed only when </body> is reached.

DOM actually built by the browser

```
body
 +-- h1
     +-- "Welcome "
     +-- p
     |   +-- "This is a paragraph "
     +-- img (src="image.jpg")       (void element, no children)
```

Effect	Result
Visual	The paragraph and image are children of the <h1>; the paragraph keeps its own default size, but the heading's semantics now wrap it.
Semantics	Screen readers announce the whole content as one heading; the document outline is wrong.
Mode	DOCTYPE is present, so the page uses **Standards Mode**, but the markup is still invalid for the validator.
Missing alt	The image is displayed, but no alternative text is available.
Cross-browser	Modern browsers produce the same DOM; older ones may differ, so do not rely on error recovery.

Q4 — Suggest a Corrected Version of the Code

Corrected, W3C-Valid HTML5

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sample</title>
</head>
<body>
  <main>
    <h1>Welcome</h1>
    <p>This is a paragraph</p>
    <img src="image.jpg" alt="Descriptive text about the image"
         width="600" height="400" loading="lazy">
  </main>
</body>
</html>
```

-
-
- Closed **</h1>** and ** **, so the heading, paragraph and image are siblings, not nested.
- Added **alt**, **width** and **height** to the image (accessibility and no layout shift).
- Added **lang**, **charset**, **viewport** and a **<main>** landmark.

Key Takeaway: <h1> and ** ** need closing tags; **<img>** is a void element that needs no closing tag but must have **alt**. Because the h1 is unclosed, the browser nests the paragraph and image inside it, so explicit closing tags are essential.

QUESTION 5 — Browser-Server Communication Flow

Sequence: (1) User enters URL → (2) DNS lookup → (3) HTTP request sent → (4) Server processes request → (5) Response returned → (6) Browser renders page.

Flow Diagram

```
Browser          DNS Server          Web Server
  | 1. types URL        |                  |
  |------- 2. DNS query --->|                |
  |<------ IP address ------|                |
  |------------ TCP handshake (+ TLS for HTTPS) ------>|
  |------------ 3. HTTP request ---------------------->|
  |                  |      4. server processes|
  |<------------ 5. HTTP response (HTML) ---------------|
  | 6. parse HTML, load CSS/JS/images, render page     |
```

Q1 — Interpret Each Step in the Communication Flow

Step	What Happens	Example
1. User enters URL	The browser parses the URL into scheme, host, port, path, query and fragment. It checks its cache and HSTS list, and may upgrade to HTTPS.	https://www.example.com/index.html
2. DNS lookup	The domain name is resolved to an IP address (details in Q3).	www.example.com → 93.184.216.34
(Hidden step) Connection setup	The browser opens a **TCP three-way handshake** (SYN, SYN-ACK, ACK) to port 80 or 443. For HTTPS a **TLS handshake** follows to negotiate encryption.	Port 443
3. HTTP request is sent	The browser sends a request line, headers and optional body.	GET /index.html HTTP/1.1 Host: www.example.com

Step	What Happens	Example
4. Server processes request	The web server routes the request, runs application code, queries databases, or reads a static file.	Nginx → PHP/Node → MySQL
5. Response is returned	The server sends a status line, headers and the body.	HTTP/1.1 200 OK Content-Type: text/html
6. Browser renders page	Parses HTML into the DOM, CSS into the CSSOM, runs JavaScript, builds the render tree, then lays out and paints. Extra resources trigger more requests.	DOM + CSSOM → render tree → layout → paint

Q2 — Identify Where Delays Can Occur

Stage	Cause of Delay
DNS lookup	Cache miss forces a chain of queries (resolver → root → TLD → authoritative); slow or distant DNS server.
TCP / TLS handshake	Each handshake costs round trips; high latency to a far server multiplies the delay.
Request transmission	Large cookies and headers, or a slow / congested network.
Server processing (TTFB)	Slow database queries, heavy application code, an overloaded server, no caching.
Response transfer	Large uncompressed HTML, images, CSS and JS; low bandwidth; packet loss.
Rendering	Render-blocking CSS/JS, heavy JavaScript, many extra requests, large images, layout shifts.
Redirects	Each redirect (e.g. http → https → www) adds a full extra round trip.

Q3 — Analyze the Role of DNS in the Process

DNS (Domain Name System) is the internet's phone book. Computers communicate using IP addresses, but humans use names, so DNS translates www.example.com into an IP address *before* any HTTP request can be sent. Without DNS the browser cannot find the server.

Lookup order (stops at the first hit):

- **Browser cache → Operating-system cache / hosts file → Recursive resolver** (ISP or 8.8.8.8) → **Root server → TLD server** (.com) → **Authoritative name server**, which returns the final IP (A / AAAA record).
- The answer is cached for the record's **TTL** (time to live), so repeat visits skip the lookup.
- DNS can also provide **load balancing** (several IPs per name) and **geo-routing** to the nearest server or CDN node.
- If DNS fails, the browser shows *DNS_PROBE_FINISHED_NXDOMAIN* or *server not found*, even if the website itself is running.

Q4 — Suggest Optimizations to Improve Performance

Area	Optimization
DNS	Use a fast DNS provider, set sensible TTLs, add <link rel="dns-prefetch"> and rel="preconnect" for third-party domains.
Connection	Use **HTTP/2 or HTTP/3** (multiplexing), keep-alive, TLS 1.3, and enable HSTS.
Distance	Serve static content through a **CDN** so it comes from a nearby edge server.
Caching	Set Cache-Control and ETag headers; use server-side caching (Redis) and a browser cache.
Server	Optimize database queries and indexes, use a faster application stack, add load balancing.
Payload	Enable **gzip / Brotli** compression; minify HTML, CSS, JS; use WebP/AVIF images and responsive sizes.
Rendering	Load scripts with defer/async, inline critical CSS, lazy-load images, set image width/height.

Requests	Remove unnecessary redirects, bundle or reduce files, remove unused code.
Measure	Use Chrome DevTools, Lighthouse and WebPageTest to find the bottleneck first.

Key Takeaway: A page load is a chain: **DNS → TCP/TLS → HTTP request → server processing → response → rendering**. DNS must succeed first because it supplies the server's IP address, and delay can occur at every link. Caching, a CDN, compression, HTTP/2+ and fewer redirects speed up the whole chain.